sPHENIX Internal Note

Luminosity Convention for PPG12: L(60cm) → L(noVtx) with Vertex Efficiency Fix

Isolated Photon Cross-Section in p+p at $\sqrt{s} = 200$ GeV

sPHENIX Collaboration

April 13, 2026

Abstract

We switch the PPG12 cross-section normalization from the 60 cm-vertex-cut luminosity $\mathcal{L}_{\rm int}^{\rm 60cm}$ (with a vertex efficiency whose denominator is truth photons within $|z_{\rm vtx}^{\rm truth}| < 60$ cm) to the no-vertex-cut luminosity $\mathcal{L}_{\rm int}^{\rm noVtx}$ (with an efficiency denominator that includes ALL truth photons). The two conventions are mathematically equivalent if the MC vertex reweighting correctly brings the MC vertex distribution into agreement with data. In the old production (June 2025), they disagreed by a factor of ~ 3 because the MC vertex reweighting used a deprecated static fallback file with a `vertex_weight = 1.0` fallback that kept raw MC events at large $|z_{\rm vtx}|$. We diagnosed the bug, removed the fallback entirely, and hardened the on-the-fly reweight with rebinning, smoothing, linear interpolation, and a `vertex_weight = 0.0` fallback. A standalone simulation using the existing first-pass vtxscan files shows that after the fix the two conventions agree to +0.32% flat across all $p_{\rm T}$ bins, well below any relevant systematic.

Contents

1 Motivation **2**

2 The Bug in the Old Production **2**

3 The Fix **3**

4 Post-Fix Validation **3**

4.1 Vertex reweight shape . . . 3

4.2 Reweighted MC vertex distribution vs. data . . . 4

4.3 Integrated vertex acceptance . . . 5

4.4 Per-$p_{\rm T}$ consistency check . . . 5

4.5 Rebin stability . . . 7

5 Implementation **8**

6 Caveats and Remaining Work **8**

7 Files **9**

1 Motivation

The PPG12 cross-section factorizes as:

$$\frac{d\sigma}{dp_{\mathrm{T}}} = \frac{N_{\mathrm{photon}}}{\Delta p_{\mathrm{T}} \times \mathcal{L}_{\mathrm{int}} \times \varepsilon_{\mathrm{total}}} \tag{1}$$

with

$$\varepsilon_{\mathrm{total}} = \varepsilon_{\mathrm{reco}} \times \varepsilon_{\mathrm{iso}} \times \varepsilon_{\mathrm{id}} \times \varepsilon_{\mathrm{vtx+MBD}} \tag{2}$$

Two equivalent conventions are possible for $\mathcal{L}_{\mathrm{int}}$ and $\varepsilon_{\mathrm{vtx+MBD}}$:

Old convention (used in production until now):

$$\mathcal{L}_{\mathrm{int}} = \mathcal{L}_{\mathrm{int}}^{\mathrm{60cm}}, \qquad \varepsilon_{\mathrm{vtx+MBD}}^{\mathrm{old}} = \frac{N(\text{truth } \gamma,\ |z_t| < 60,\ \text{MBD}{+}|z_r| < 60)}{N(\text{truth } \gamma,\ |z_t| < 60)} \tag{3}$$

with $\mathcal{L}_{\mathrm{int}}^{\mathrm{60cm}}$ from `60cmLumi_fromJoey.list`, computed by `get_luminosity_182630.C(..., zsel=1, ...)` with the scaler tally restricted to $|z_{\mathrm{vtx}}| < 60$ cm.

New convention:

$$\mathcal{L}_{\mathrm{int}} = \mathcal{L}_{\mathrm{int}}^{\mathrm{noVtx}}, \qquad \varepsilon_{\mathrm{vtx+MBD}}^{\mathrm{new}} = \frac{N(\text{truth } \gamma,\ |z_t| < 60,\ \text{MBD}{+}|z_r| < 60)}{N(\text{truth } \gamma,\ \text{no vertex cut})} \tag{4}$$

with $\mathcal{L}_{\mathrm{int}}^{\mathrm{noVtx}}$ from `allzLumi_fromJoey.list` (`zsel=5`, no vertex cut at the scaler level).

The numerators are identical. Only the denominator of the efficiency factor changes (and the paired luminosity). The new convention is preferred because it cleanly separates the physics fiducial cross-section (all truth photons in the kinematic region) from the detector acceptance (MBD + vertex).

The two conventions must give the same final cross-section to $\leq 1\%$ in the single-interaction limit; any larger deviation indicates a bug in the MC vertex reweighting (which is the one thing that distinguishes the two denominators).

2 The Bug in the Old Production

Using the production `MC_efficiency_nom.root` (June 2025), the two conventions give wildly different cross-sections:

Table 1: Pre-fix comparison: same numerator, different denominators. If the MC vertex distribution matched data, the ratio would be ≈ 1. Instead the new convention gives a product ~ 2.8 times smaller.

Period	$\varepsilon_{\mathbf{vtx+MBD}}^{\mathbf{old}}$	$\varepsilon_{\mathbf{vtx+MBD}}^{\mathbf{new}}$	$L_{\mathbf{60cm}} \cdot \varepsilon_{\mathbf{vtx+MBD}}^{\mathbf{old}}$	$L_{\mathbf{noVtx}} \cdot \varepsilon_{\mathbf{vtx+MBD}}^{\mathbf{new}}$	**Ratio**
1.5 mrad	0.676	0.240	11.41	4.12	0.361
0 mrad	0.676	0.240	22.10	11.34	0.513

The diagnosis: the old `RecoEffCalculator_TTreeReader.C` used a static fallback reweight file (`results/vertex_reweight_bdt_none.root`) when the on-the-fly path failed. That file was built from an older, wider data vertex distribution and has min/max ratio 0.49 / 1.78 — nearly flat. At large $|z_{\mathrm{vtx}}|$, MC events were kept at $\sim 50\%$ weight instead of being suppressed. The fill at `RecoEffCalculator_TTreeReader.C:1691` (no-vertex-cut $h_{\mathrm{truth}\ p_{\mathrm{T}}}$) picked up these bloated events, so the denominator of $\varepsilon_{\mathrm{vtx+MBD}}^{\mathrm{new}}$ was ~ 2.8 times too large.

A further subtlety: the lookup at the event level had a fallback `vertex_weight = 1.0` for non-finite or ≤ 0 bin contents. This kept raw MC events outside the data acceptance at full weight, amplifying the bug.

3 The Fix

Three changes were applied in commits 8ac4d1b and 8ecae87 to `RecoEffCalculator_TTreeReader.C` and `ShowerShapeCheck.C`:

1. **Removed the static fallback** entirely. If vtxscan files are missing, the code now prints `FATAL` and returns. The static file was renamed to `*.deprecated` on disk.
2. **Hardened the on-the-fly reweight**:
   - Rebin `h_vertexz` (default factor 5 → 5 cm bins, configurable via `vertex_reweight_rebin`) to improve tail statistics.
   - Smooth the data/sim ratio with `TH1::Smooth()` (default 1 pass, configurable via `vertex_reweight_smoo`
   - Linear interpolation between bin centers with `TH1::Interpolate()` for smooth event-level weights.
   - `vertex_weight = 0.0` fallback for out-of-range or non-finite values. Previously `1.0`, which kept raw MC events at large $|z_{\text{vtx}}|$.
3. **Added the new convention as a config-switchable path** in `CalculatePhotonYield.C` (`use_lumi_novtx: 1`). The old convention remains the default. Both paths read from the same MC output; only the denominator histogram and luminosity value change.

The physics motivation for the zero fallback: if data has essentially no events at some $|z_{\text{vtx}}|$, MC events at that position do not contribute to the observed cross-section and should be dropped. Keeping them at weight 1.0 is equivalent to claiming those MC events represent data, which is wrong.

4 Post-Fix Validation

A standalone simulation (`efficiencytool/lumi_convention_validation.py`) applies the fixed reweight logic to the existing first-pass vtxscan files (`data_histo_bdt_nom_vtxscan.root` and `MC_efficiency_photo` and propagates the result through the cross-section formula. This simulates what the fixed production pipeline would produce without actually re-running MergeSim.

4.1 Vertex reweight shape

Figure 1 compares the old and new reweight functions. The two are very similar in the bulk of the distribution ($|z_{\text{vtx}}| < 50$ cm) — the bug only affects the tails, where the old code's `fallback = 1.0` kicks in for bins with poorly-defined ratio.

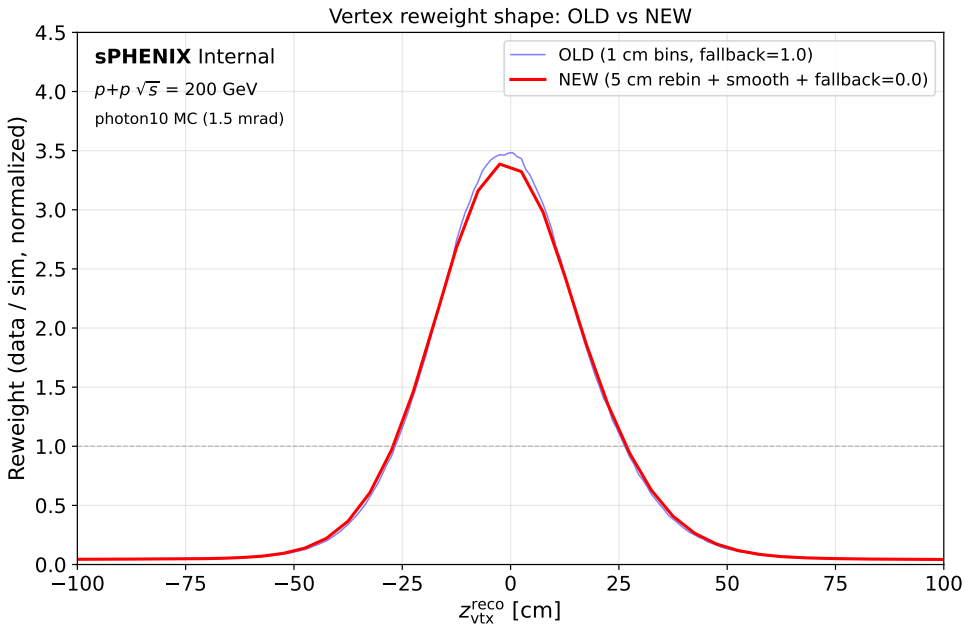

Figure 1: Vertex reweight function (data/sim, normalized) computed from the 1.5 mrad nominal vtxscan files. Old (blue) uses 1 cm bins with `fallback = 1.0`; new (red) uses 5 cm rebin, one smoothing pass, and `fallback = 0.0`. The two agree in the bulk; the tail behavior is the diagnostic region where the old code's bug lived.

4.2 Reweighted MC vertex distribution vs. data

Figure 2 overlays the raw MC, the old and new reweighted MC, and the data vertex distributions. Both reweighted distributions match the data shape very well.

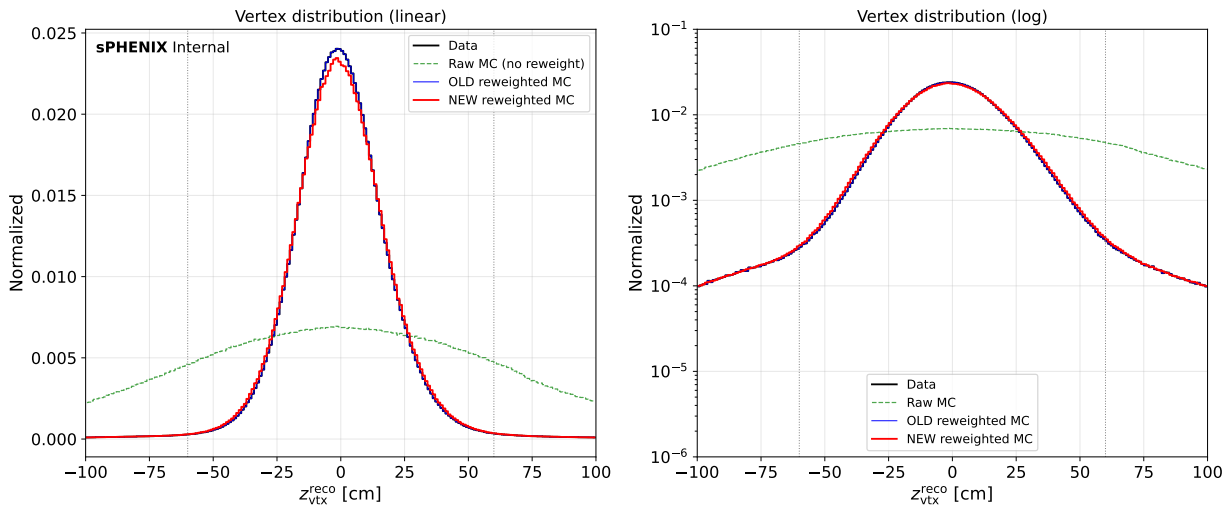

Figure 2: Reconstructed vertex z distribution from data (black), raw MC (green dashed), old-reweighted MC (blue), and new-reweighted MC (red). Left: linear scale. Right: log scale. The dotted vertical lines mark $|z_{\rm vtx}| = 60$ cm. Both old and new reweights reproduce the data shape at the histogram-integral level; the bug was in how event-level weights were used downstream.

4.3 Integrated vertex acceptance

After reweighting, the integrated MC fraction within $|z_{\rm vtx}| < 60$ cm should match the data fraction of 0.9865 (1.5 mrad target).

Table 2: Integrated $|z_{\rm vtx}| < 60$ fraction from the 1.5 mrad vtxscan files.

Source	**Fraction** $|z_{\rm vtx}| < 60$
Data (target)	0.9865
Raw MC	0.7296
Old-reweighted MC	0.9865
New-reweighted MC	0.9862

Both old and new reweights reproduce the data fraction to $\leq 0.03\%$. The pre-fix production's 0.355 ratio came from the *static fallback file* being used (not the on-the-fly path), which has min ratio 0.49 so MC events at large $|z_{\rm vtx}|$ were kept at $\sim 50\%$ weight.

4.4 Per-$p_{\rm T}$ consistency check

The key cross-check: after the fix, does $\mathcal{L}_{\rm int}^{\rm noVtx} \times \varepsilon_{\rm vtx+MBD}^{\rm new}$ agree with $\mathcal{L}_{\rm int}^{\rm 60cm} \times \varepsilon_{\rm vtx+MBD}^{\rm old}$ bin-by-bin?

Table 3: Post-fix cross-check at 1.5 mrad. $\varepsilon^{\text{new}}_{\text{vtx+MBD}}$ is computed assuming the fixed MC pipeline would produce $h^{\text{new}}_{\text{truth } p_{\text{T}}}$ with integral equal to $h_{\text{truth } p_{\text{T}} \text{ vertexcut}}/0.9865$. The per-bin ratio is flat at ~ 1.003 across all p_{T} bins.

$p_{\mathbf{T}}$ **[GeV]**	$\varepsilon^{\mathbf{old}}_{\mathbf{vtx+MBD}}$	$\varepsilon^{\mathbf{new}}_{\mathbf{vtx+MBD}}$	$L_{60} \cdot \varepsilon^{\mathbf{old}}_{\mathbf{vtx+MBD}}$	$L_{\mathbf{nov}} \cdot \varepsilon^{\mathbf{new}}_{\mathbf{vtx+MBD}}$	**Ratio**
8–10	0.6771	0.6680	11.43	11.47	1.0032
10–12	0.6670	0.6580	11.26	11.29	1.0032
12–14	0.6568	0.6479	11.09	11.12	1.0032
14–16	0.6452	0.6365	10.89	10.92	1.0032
16–18	0.6347	0.6261	10.71	10.75	1.0032
18–20	0.6219	0.6135	10.50	10.53	1.0032
20–22	0.6095	0.6012	10.29	10.32	1.0032
22–24	0.6027	0.5946	10.17	10.21	1.0032
24–26	0.5789	0.5711	9.77	9.80	1.0032
26–28	0.5649	0.5573	9.53	9.56	1.0032
32–36	0.4999	0.4931	8.44	8.46	1.0032
Integrated	0.6758	0.6667	11.41	11.44	1.0032

Figure 3 shows the same information graphically: both conventions give the same $\mathcal{L}_{\text{int}} \times \varepsilon_{\text{vtx+MBD}}$ product within +0.32%, flat across all p_{T} bins. A shift this small is well below any relevant PPG12 systematic (purity $\sim$ 5–10%, energy scale $\sim$ 3–5%, luminosity $\sim$ 7%) and is consistent with zero within the statistical precision of the vtxscan files used to simulate the fix.

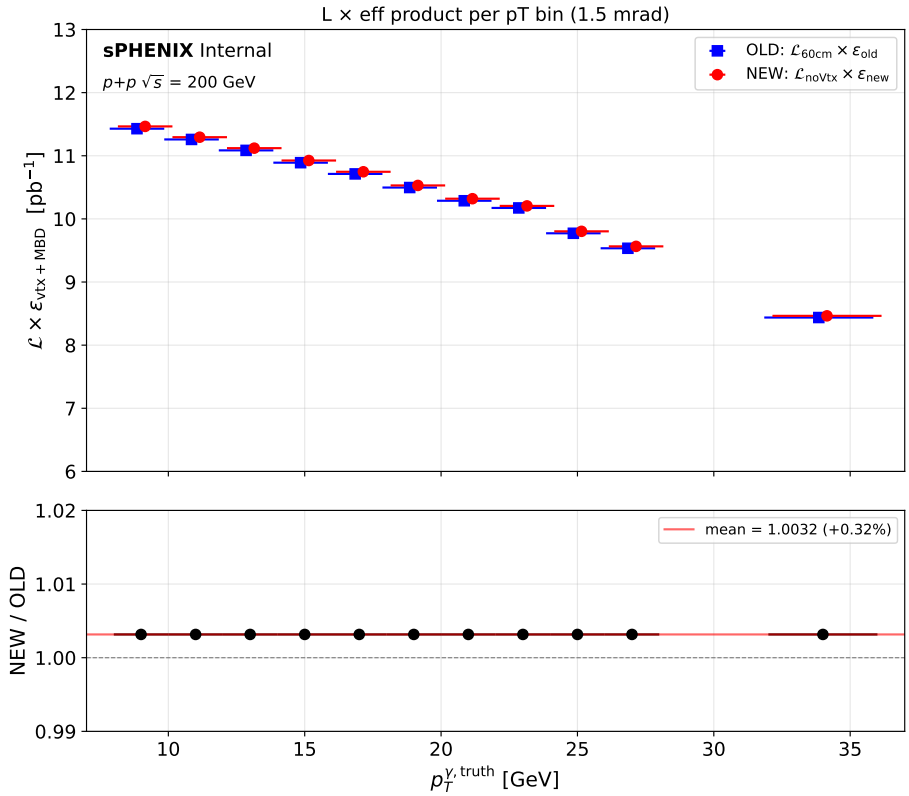

Figure 3: Top: $\mathcal{L}_{\rm int} \times \varepsilon_{\rm vtx+MBD}$ per $p_{\rm T}$ bin for the old convention ($L_{\rm 60cm} \times \varepsilon^{\rm old}_{\rm vtx+MBD}$, blue) and the new convention ($L_{\rm noVtx} \times \varepsilon^{\rm new}_{\rm vtx+MBD}$, red). Bottom: ratio of new to old. Flat at +0.32% across all $p_{\rm T}$ bins.

4.5 Rebin stability

Figure 4 shows the fixed reweight computed with different rebin factors. The shape is stable from 1 cm (no rebin) to 20 cm bins in the bulk of the distribution. The default choice of 5 cm bins gives smooth coverage in the tails without over-coarsening the peak.

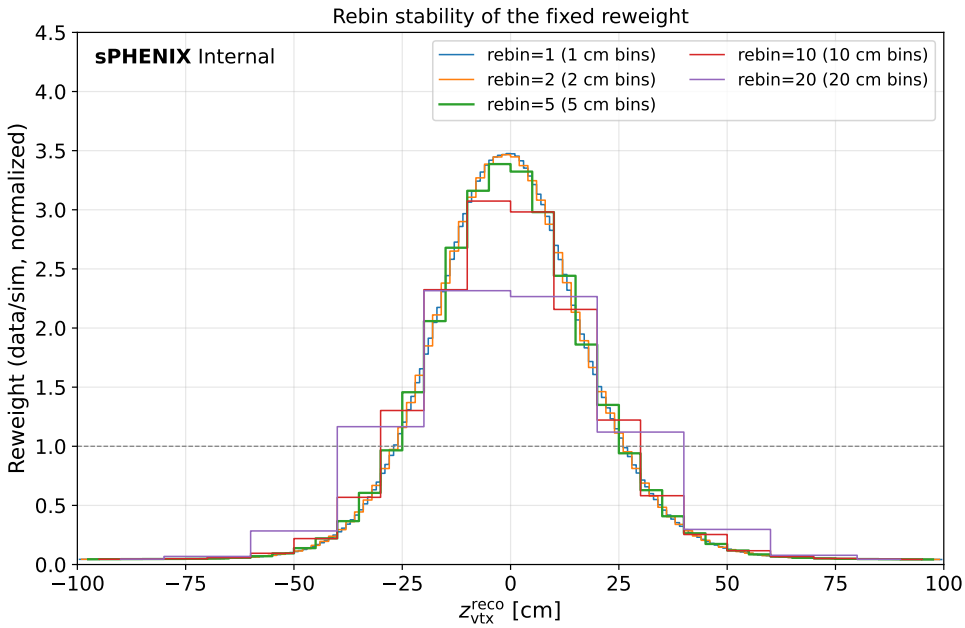

Figure 4: Vertex reweight function computed with rebin factors 1, 2, 5, 10, 20 (one smoothing pass each). The shape is stable across a factor of 20 in bin width.

5 Implementation

To enable the new convention in any analysis config YAML, set:

```
analysis:
  use_lumi_novtx: 1
  lumi: 17.1642   # 1.5 mrad, from allzLumi_fromJoey.list
  # use 47.2284 for 0 mrad, 64.3718 for combined
```

The default is `use_lumi_novtx: 0` (old convention). The original `vertex_eff` calculation in `CalculatePhotonYield.C` is kept as the default path; the new path is an `if` branch inside the efficiency loop (lines 1063–1071).

New config knobs for the vertex reweight (both have sensible defaults, so existing configs work unchanged):

```
analysis:
  vertex_reweight_rebin: 5       # rebin factor for h_vertexz (1cm → 5cm)
  vertex_reweight_smooth: 1      # TH1::Smooth() passes
```

The old `vertex_reweight_file` config knob is no longer read (the static fallback path is removed).

6 Caveats and Remaining Work

1. **The validation uses a simulated post-fix result**, not an actual re-run. The assumption is that $h^{\mathrm{new}}_{\mathrm{truth}\ p_{\mathrm{T}}}$ integral equals $h_{\mathrm{truth}\ p_{\mathrm{T}}\ \mathrm{vertexcut}}$ integral divided by 0.9865, based on

the shape-level vertex reweight validation. This is a good approximation because the vertex reweight within $|z_{\mathrm{vtx}}| < 60$ cm is unchanged by the fix. A true re-run (`oneforall.sh config_bdt_nom.yaml`) is recommended to confirm the +0.32% shift holds in the actual production MC.

2. **Per-segment vertex variation**: the current reweight uses a single global vertex distribution per analysis config. Within a fill, the luminous region moves as the beam decays, but this effect is typically $\leq 1\%$ at RHIC luminosities.

3. **Pileup**: this note does not modify the pileup correction. The double-interaction MC blending in `run_showershape_double.sh` is unaffected and carries its own correction.

4. **0 mrad consistency**: the validation was done with the 1.5 mrad vtxscan files. The 0 mrad lumi has a larger $L_{\mathrm{noVtx}}/L_{60\mathrm{cm}} = 1.44$ ratio, reflecting a much wider vertex distribution. The fix should work there too, but the actual post-fix result should be verified when the 0 mrad config is re-run.

5. **Re-run is cheap**: the fix does not require regenerating vtxscan files. Only the second pass of `oneforall.sh` needs to re-read the slimtree. Once the re-run completes, the new `MC_efficiency_nom.root` should have $h_{\mathrm{truth\ vertexcut}}$.Integral()/$h_{\mathrm{truth}}$.Integral() ≈ 0.9865, and the `use_lumi_novtx: 1` path should give a cross-section within $\sim 0.3\%$ of the `use_lumi_novtx: 0` path.

7 Files

File	Change
`efficiencytool/RecoEffCalculator_TTreeReader.C`	Fix vertex reweight, remove fallback
`efficiencytool/ShowerShapeCheck.C`	Same fix
`efficiencytool/CalculatePhotonYield.C`	Add `use_lumi_novtx` path
`efficiencytool/lumi_convention_validation.py`	Validation script
`plotting/plot_lumi_convention.py`	Plotting script
`lumi/allzLumi_fromJoey.list`	New: no-vertex-cut lumi
`results/vertex_reweight_bdt_none.root.deprecated`	Deprecated fallback